

Don't Forget the Comma in Single-Item Tuples

When writing tuple values in your code, keep in mind that you'll still need a trailing comma even if the tuple only contains a single item. Although the value `(42, )` is a tuple that contains the integer `42`, the value `(42)` is simply the integer `42`. The parentheses in `(42)` are similar to those used in the expression `(20 + 1) * 2`, which evaluates to the integer value `42`. Forgetting the comma can lead to this:

```
>>> spam = ('cat', 'dog', 'moose')
>>> spam[0]
'cat'
>>> spam = ('cat')
❶ >>> spam[0]
'c'
❷ >>> spam = ('cat', )
>>> spam[0]
'cat'
```

Without a comma, `('cat')` evaluates to the string value, which is why `spam[0]` evaluates to the first character of the string, `'c'` ❶. The trailing comma is required for the parentheses to be recognized as a tuple value ❷. In Python, the commas make a tuple more than the parentheses.

Summary

Miscommunication happens in every language, even in programming languages. Python has a few gotchas that can trap the unwary. Even if they rarely come up, it's best to know about them so you can quickly recognize and debug the problems they can cause.

Although it's possible to add or remove items from a list while iterating over that list, it's a potential source of bugs. It's much safer to iterate over a copy of the list and then make changes to the original. When you do make copies of a list (or any other mutable object), remember that assignment statements copy only the reference to the object, not the actual object. You